CSC 3320: System-Level Programming

Chapter 1: Introduction (contd.)
Chapter 2: C Fundamentals

Instructor: Dr. Md Mahfuzur Rahman
Department of Computer Science, GSU

Note: These slides are updated based on original notes from Raj Sunderraman, Michael Weeks, and Yuan Long

Properties of C

Low-level

- Suitable language for systems programming

Small

- Relies on a "library" of standard functions

Permissive

- It assumes that you know what you're doing, so it allows you a wider degree of latitude than many languages. It doesn't mandate the detailed error-checking found in other language
  - E.g. C has **no array index out of bounds** error.

    So be careful when you use index in an array.

Strengths of C

Efficiency

- Intended for applications where assembly language had traditionally been used.

Portability

- Hasn't splintered into incompatible dialects; small and easily written.

Power

- Large collection of data types and operators.

Flexibility

- Not only for system but also for embedded system commercial data processing

Standard library.

- Functions for input/output, string handing, storage allocation, and other useful operations.

Integration with UNIX

Weaknesses of C

Error-prone

- Programming mistakes that would be caught in many other languages cannot be detected by a C compiler.

Difficult to understand

Difficult to modify

First C program

pun.c

```
#include <stdio.h>
/* My first C program which prints a pun */
int main(void)
{
  printf("To C, or not to C: that is the question.\n");
  return 0;
}
```

Pre-processor, import I/O library

Comments

main() means "start here"

Function from standard I/O library

New line

Return 0 from main means our program finished without errors

- This program might be stored in a file named pun.c.
- The file name doesn't matter, but the .c extension is often required.

Compiling and Linking

Before a program can be executed, three steps are usually necessary:

- ***Preprocessing.*** The ***preprocessor*** obeys commands that begin with # (known as ***directives***)
- ***Compiling.*** A ***compiler*** translates then translates the program into machine instructions (***object code***).
- ***Linking.*** A ***linker*** combines the object code produced by the compiler with any additional code needed to yield a complete executable program.
  - E.g. Link ***printf*** from library

The preprocessor is usually integrated with the compiler.

The GCC Compiler

GCC is one of the most popular C compilers.

GCC is supplied with Linux but is available for many other platforms as well.

Using this compiler is similar to using `cc`:

```
% gcc -o pun pun.c
```

The General Form of a Simple Program

Simple C programs have the form

directives

```
int main(void)
{
        statements
}
```

Functions

Directives

Before a C program is compiled, it is first edited by a preprocessor.

Commands intended for the preprocessor are called directives.

Example:

```
#include <stdio.h>
```

- `<stdio.h>` is a ***header*** containing information about C's standard I/O library.

Directives

Directives always begin with a # character.

By default, directives are one line long; there's no semicolon or other special marker at the end.

awk command

- It's a powerful text processing tool

Imagine a text file containing student names and gpa:

Rahman Mahfuzur 3.5

Symonds Andrew 4
Patel Krish 2.5
Akter Sharmin 3
Mahfuz Anandita 3.25
Chowdhury Shahriar 3.1
Mahfuz Arady 2
Issue this command:

```
awk '{print $3, $1, $2}' students_gpa.txt
```

Conditional awk

```
awk '$3>3 {print $3, $2}' students_gpa.txt
```

Output:

3.5 Mahfuzur

4 Andrew

3.25 Anandita

3.1 Shahriar

awk + regex

```
awk '/fuz/ {print $3, $2}' students_gpa.txt
```

Output:

3.5 Mahfuzur

3.25 Anandita

2 Arady

awk command

Save this awk command in a file:

```
BEGIN {FS = ","
print "Students Name and GPA:"
print "-----------------------------------"}
        {print $1}
END {print "-------------------------"}
```

Issue this command:

```
awk -f awk_parser.awk
students_gpa.csv
```

awk command example

```
BEGIN {FS=","
print "Students Name and GPA:"
print "---------------------------"}
$3>3  {print $1,$3}
/fuz/ {print $1,$3}
END {print "---------------------"}
```

Output:

```
Students Name and GPA:
_____________________________
Rahman 3.5
Rahman 3.5
Symonds 4
Mahfuz 3.25
Mahfuz 3.25
Chowdhury 3.1
Mahfuz 2
--------------------
```

For loop

```
#!/bin/bash
# understanding how for loop works
for count in {1..10..2}
do
  echo $count
  # mkdir dir$count
  rm -r dir$count
done
```